



VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity

Qianxi Zhang¹ Shuotao Xu¹ Qi Chen^{1,*} Guoxin Sui¹ Jiadong Xie^{1,2} Zhizhen Cai^{1,3}
Yaoqi Chen^{1,3} Yinxuan He^{1,4} Yuqing Yang¹ Fan Yang¹ Mao Yang¹ Lidong Zhou¹

¹Microsoft Research Asia

²East China Normal University

³University of Science and Technology of China

⁴Renmin University of China

Abstract

Approximate similarity queries on high-dimensional vector indices have become the cornerstone for many critical online services. An increasing need for more sophisticated vector queries requires integrating vector search systems with relational databases. However, high-dimensional vector indices do not exhibit monotonicity, a critical property of conventional indices. The lack of monotonicity forces existing vector systems to rely on monotonicity-preserving tentative indices, set up temporarily for a target vector's TopK nearest neighbors, to facilitate queries. This leads to suboptimal performance due to the difficulty to predict the optimal K .

This paper presents VBASE, a system that efficiently supports complex queries of both approximate similarity search and relational operators. VBASE identifies a common property, *relaxed monotonicity*, to unify two seemingly incompatible systems. This common property allows VBASE to circumvent the constraints of a TopK-only interface to achieve significantly higher efficiency, while provably preserving the semantics of TopK-based solutions. Evaluation results show VBASE offers up to three orders-of-magnitude higher performance than state-of-the-art vector systems on complex online vector queries. VBASE further enables analytical similarity queries that previous vector systems do not, and shows $7,000\times$ speedup with 99.9% accuracy of exact queries.

1 Introduction

Recent advances in deep learning (embedding) models map almost all types of data (e.g., images, videos, documents) into high-dimension *vectors* [60, 66, 88]. Queries on high-dimensional vectors enable complex semantic-analysis that was previously difficult if not impossible, thus they become the cornerstone for many important online services like search [25, 51], eCommerce [54], and recommendation systems [49, 53, 56, 84]. The “online” nature of these

services requires vector search to complete in milliseconds [31, 36, 42, 64]. Such a strict latency conflicts with the inherently high cost of exact search algorithm [28], which forces end-users to settle on approximate query results on high-dimensional vectors. With emerging new vector search applications, queries on vectors become increasingly more complex, which often involve hybrid search on both scalar and vector data (§2.1). This naturally motivates an integration of vector search systems and relational databases.

Vector search and database systems differ in their ways of using the index, a critical structure to speed up queries. An important property of conventional indices like B-tree [22] is *monotonicity*. This property ensures that a query can traverse the data-set guided by an index monotonically along a certain direction. This often avoids total data scan, therefore enables efficient query execution. However, it is prohibitively expensive for high-dimensional vector indices [5, 25, 43, 55, 57, 85] to preserve monotonicity, because of the curse of dimensionality [28]. Instead, they are often organized as a graph or cluster-based irregular structure, which follows monotonicity *approximately*. Traversing such vector indices does not guarantee a strict monotonic order in terms of distances to a target vector, but it enables a system to efficiently determine when it is *unlikely* for new traversals to reach closer vectors to a target than the current K ones. Therefore, modern vector indices only support approximate TopK, i.e., to find K nearest neighbors approximately. A TopK query traverses a vector index for a sufficiently large number of steps, until it determines that a neighbor closer than the current K nearest ones is unlikely to be found.

To integrate vector search and database systems, existing vector database systems [76, 80, 86] choose to conform with *strict* monotonicity. To support similarity queries other than TopK, they first leverage TopK to collect K vectors, and sort the K vectors according to distances to a target vector, which sets up a *temporary index preserving monotonicity*. Complex relational operators can therefore execute on the temporary index in the traditional way. Consider the following vector search query, “find X number of products most similar to an

*Corresponding author.

image but under a certain price”. A database planner would first run a vector search operator on the vector attribute of image embedding to find K nearest tuples, then apply a filter operator on the price attribute. But it is impossible to predict exactly how many tuples will pass the filter operator, which could be much less than K . Therefore this practice has the inherent difficulty of identifying the optimal K that produces exact X results. As a result, it resorts to either a setting of a conservatively large K or a trial-and-error of many K s, which both lead to suboptimal query performance.

In this paper, we present VBASE, a new system capable of efficiently serving complex online queries that involve both approximate similarity search and relational operators on scalar and vector data-sets. VBASE identifies *Relaxed Monotonicity* as the common property abstracted from the two seemingly different systems: vector search systems and relational databases. *Relaxed monotonicity* requires index traversals to only follow monotonicity *approximately*. We observe that state-of-the-art vector indices all follow relaxed monotonicity property in a two-phase pattern: an index traversal first locates the nearest region to a target vector approximately, and then moves away from the target region progressively in an approximate way. Based on the observation, we formally define *Relaxed Monotonicity* property, which abstracts the core index traversal pattern that most existing vector indices already have (§3.1). *Relaxed monotonicity* can be viewed as a generalized form of monotonicity, thus it is also applicable to conventional scalar indices, such as B-trees. Therefore, *Relaxed Monotonicity* can serve as the common foundation of vector search and database systems.

With relaxed monotonicity, VBASE builds a unified query execution engine to support a wide range of queries both on scalar and vector data, including queries across multiple heterogeneous indices. VBASE’s unified engine is based on a Next interface, instead of TopK, to support traversal in both vector and scalar indices. Meanwhile, the engine allows the derivation of a generalized termination condition from relaxed monotonicity to stop a query’s execution timely.

A unique characteristic of VBASE is that its relaxed-monotonicity-based query execution engine can *provably* achieve *equivalent* query results to those produced by TopK-only solutions of the *optimal* $\tilde{K}$ (§3.3). This powerful property allows VBASE to circumvent the constraints of a TopK-only interface to achieve significantly higher efficiency, while preserving the semantics of TopK-based queries. In particular, based on the derived generalized termination condition, VBASE is able to detect the $\tilde{K}$ during index traversal without an prediction of $\tilde{K}$. This allows VBASE to achieve similar performance to the well-optimized TopK vector search. For more complex queries than TopK searches, VBASE can achieve up to three order-of-magnitude lower average and tail query latency over state-of-the-art systems under similar result accuracy (i.e., same or even better recalls).

Moreover, with relaxed monotonicity, VBASE can even

Table 1: Online Similarity Query Support for Vectors

	S1	S2	S3	S4
ANN systems [25,43,46]	✓	✗	✗	✗
AnalyticDB-V [80]	✓	✓	✗*	✗*
PASE [86]	✓	✓	✗*	✗*
PostgreSQL [12]	✗*	✗*	✗*	✗*
Milvus [76]	✓	✓	✓	✗
Elasticsearch [4]	✓	✓	✗ ^o	✗

*: Some systems can support these queries through exhaustive linear scan, but this cannot meet the requirements of online services.

^o: Only support one inverted index and one vector index.

support approximate query types that previous systems do not, and show superior query performance and accuracy. For example, VBASE can finish a join-based vector query in 16 seconds with 99.9%+ recall rate, which is 7000× faster than a brute-force table scan.

In summary, we make the following contributions:

1. VBASE identifies and defines formally “*Relaxed Monotonicity*”, a property that reveals, for the first time, the core of well-designed vector indices and why they work effectively in practice.
2. VBASE builds a *Unified Database Engine* based on *relaxed monotonicity*, which enables powerful complex queries leveraging both vector and scalar data indices.
3. We prove that VBASE’s unified engine produces equivalent results to TopK-only methods using vector indices, with a much more efficient execution plan than that of TopK-only methods.
4. We implement VBASE based on PostgreSQL with 2000 lines of additional code, and show an end-to-end evaluation of eight complex SQL queries on a hybrid one million recipe data-sets [59] with both vector and scalar attributes.

We plan to make VBASE open-source to satisfy the emerging important vector analytic applications in the era of AI.

2 Background

2.1 Emerging Online Vector Queries

Vector has become a key form of data representation in the AI era. Deep learning has enabled a growing number of vector-centric online applications, including embedding-based retrieval [25,87], face recognition [69], code retrieval [37], question answering [52,63], Google Multisearch [7], Facebook near-exact duplicates detection [6], etc. More recently, AI applications have leveraged ChatGPT’s retrieval plugin [10] to convert their proprietary knowledge, personal documents, and chat contexts into vectors. This enables the retrieval of relevant vectors with price, category, location, or time constraints to construct prompts in the chat.

Traditional applications also benefit from vectors empowered by AI. For example, search engine turns web documents into both bag-of-words sparse vectors and deep learning em-